

PilotFish Database (SQL) Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This transport turns **XML in the transaction body** into **SQL against your database** using PilotFish's SQLXML engine — inserts, updates, and queries driven by an input file or inline SQL. It is the outbound/database-write counterpart to the Database SQL processor family.

Database (SQL) Transport

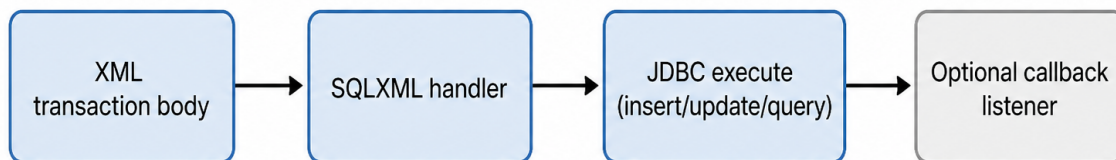


Figure 1: At a glance (plain English)

- **Executes** SQLXML documents (from file or generated simple SQL) via JDBC.
- **Connects** through JNDI Data Source or direct JDBC URL + driver.
- **Supports** connection pooling, timeouts, transaction isolation, and callback listeners for result sets.

- **Overrides `sendData`** when pooling is enabled to manage completion state explicitly.

Purpose

This document is a **deep dive** into the **Database (SQL)** transport as shipped in **eip.war.hs.26R1.11**: delegation to `DatabaseSqlExecution`, SQLXML execution, pooling vs single-handler modes, and callback delivery.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-db-1.0.1.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.db.DatabaseSqlTr</code>
Execution helper	<code>com.pilotfish.eip.modules.db.DatabaseSqlEx</code>
SQLXML engine	<code>com.pilotfish.eip.util.SQLXMLHandler</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../Da</code>
Transport type string	Database (SQL)
Description	Translates incoming xml data into SQL statements which are used to populate/update a specified database.
Help ID	<code>console.transport.databasesql</code>
Categories	SQL, DATABASE

1. Conceptual model

Database (SQL) transport is a thin **Transport façade** over `DatabaseSqlExecution`, which is shared with Database SQL processor patterns. The transaction body is treated as **SQLXML input** (or replaced by generated SQLXML when **Write Query** is enabled).

Transaction XML body

```
|
v
[Write Query?] --> buildSimpleSQLXML (Query + QueryParams)
|
v
SQLXMLHandler (JDBC connection)
|
v
handler.execute(data) -- SQLXML interprets XML -> SQL
|
v
[CallbackListenerName set?] --> trigger listener with result stream
```

Implements `SQLXMLModule`, `AbstractReferencedListeners`, and `AbstractConfiguration`.

2. High-level behavior

2.1 sendData dispatch

From `DatabaseSqlTransport.sendData`:

1. `checkRequiredData(data)`
2. If `UseConnectionPooling = true` → call `executeTransport` directly (bypasses `AbstractTransport.sendData` wrapper timing).
3. Else → `super.sendData(data)` (standard transport error/complete handling).
4. On any `Throwable`: `setTransactionStateError()`, wrap in `EIPEException("Error in DatabaseSqlTransport Transport")`.
5. Always ends with `setTransactionStateComplete()` on success path (including pooled path).

2.2 executeTransport / executeSQL

Delegated to `DatabaseSqlExecution.executeSQL`:

1. If **Write Query** → `buildSimpleSQLXML` replaces body with generated SQLXML from **SQL + Query Params** table.
2. `getHandler(data, module)` — pooled or singleton `SQLXMLHandler` (reinitialized per config).
3. `handler.execute(data)` — core SQLXML interpretation.
4. `returnHandler(handler)` when pooling.
5. If **CallBack listener** configured → `TriggerableListenerUtils.trigger` with post-execution stream.

`systemShutdown()` → `sqlExecution.shutdownHandler()` closes JDBC resources.

3. Configuration reference

Configuration is defined in `DatabaseSqlExecution.getConfigurationDescriptor` (title `DatabaseSqlConfiguration`).

3.1 Input mode

UI label	Tag	Type	Default	Nuances
Write Query	<code>WriteQuery</code>	Boolean	<code>false</code>	When true, uses SQL + params instead of Input file
Input file	<code>InputFile</code>	File	<code>""</code>	SQLXML template; visible when Write Query false
SQL	<code>Query</code>	Text	<code>""</code>	Simple SQL when Write Query true
Query Params	<code>QueryParams</code>	Table	<code>empty</code>	Ordered bind parameters

3.2 Connection (tab: Connection)

UI label	Tag	Type	Default	Required*
Type	UseDataSource	Trigger	Data Source	—
User name	UserName	String	""	JDBC mode
Password	Password	Password	""	JDBC mode
Data Source	DataSource	DataSource	""	Data Source mode
JDBC driver	JdbcDriver	JDBCdriver	""	JDBC mode
JDBC URL	JdbcURL	String	""	JDBC mode
Test Connection	<i>(button)</i>	—	—	Action TestSqlConnection

* Completeness: Data Source **or** (driver + URL) required (isConfigurationComplete).

3.3 Advanced execution

UI label	Tag	Default	Nuances
Keep connection	KeepConnection	false	JDBC mode only; reuse between transports
Autocommit Transactions	Autocommit	false	Each SQLXML doc = transaction when false
Enable Timeout for Queries	EnableTimeout	false	Gates timeout field
Timeout for Queries	Timeout	600	Seconds; aborts long queries
Disable Metadata	DisableMetadata	false	Skips DB metadata load
Restrict Metadata To Catalog	RestrictMetadataToCatalog	true	Performance filter
Restrict Metadata To Schema	RestrictMetadataToSchema	true	Performance filter
Restrict Metadata To Table(s)	RestrictMetadataToTable	true	Table pattern
CallBack listener	CallbackListenerName	""	Programmable (Trigger)
Use Single Output Stream	UseSingleOutputStream	false	Single combined result stream
Error on Unknown Elements	ErrorOnUnknownElement	false	Strict SQLXML
Use JDBC Identity for Inserts	UseJdbcGeneratedKeys	true	Overrides SQLXML IdentityQuery

3.4 Pooling (tab: Pooling)

UI label	Tag	Default	Nuances
Use Connection Pooling	UseConnectionPooling	true	Changes sendData dispatch

UI label	Tag	Default	Nuances
Connections allowed	<code>PermittedDBConnections</code>	<code>5</code>	Max concurrent handlers

3.5 JDBC properties & isolation

UI label	Tag	Tab
JDBC Properties	<code>JDBCProperties</code>	JDBC Props
Use JDBC Parameter Typing	<code>UseJdbcTyping</code>	Compatibility
Transaction Isolation	<code>TransactionIsolationLevel</code>	Transaction Isolation
Driver Specific Level	<code>TransactionIsolationDriverSpecific</code>	Driver Specific Isolation

3.6 Debug

UI label	Tag	Default
Log Metadata	<code>LogMetadata</code>	<code>false</code>
SQLXML Logging	<code>LogSQL</code>	<code>false</code>

3.7 Inherited AbstractTransport

Standard transport module binding; no extra conditional flags beyond configuration completeness.

4. Runtime algorithms

4.1 Handler acquisition (non-pooled)

- Single `_handler` instance per transport module instance, **reinitialized** each execution with current config.
- Applies timeout, metadata constraints, autocommit, generated-keys, single-output flags before return.

4.2 Handler acquisition (pooled)

- Keyed pool per connection signature string.
- Spins with `Thread.sleep(100)` until slot available under **Connections allowed**.
- Handlers pushed/popped from per-key `LinkedList`; `currentHandlersInUse` tracked.

4.3 Simple SQLXML generation

`CommonDatabaseUtils.buildSimpleSqlXmlQuery(sql, params) → DOM → transform to bytes → data.setDataStream.`

4.4 Callback

After successful execute, if `CallbackListenerName` non-empty, triggers with current `data.getDataStream()`.

5. Transaction attributes

No attributes registered directly on `DatabaseSqlTransport`. SQLXML execution may set attributes per SQLXML template semantics (handler-dependent).

6. Worked example — pooled insert transport

Configuration sketch:

- **Use Connection Pooling:** enabled, 10 connections
- **JDBC URL:** `jdbc:postgresql://db:5432/claims`
- **Input file:** `insert-claim.sqlxml`
- **Autocommit:** false (one SQLXML document = one DB transaction)

Each transport invocation borrows a handler, executes XML body against JDBC, returns handler to pool, marks transaction complete. Under load, eleventh concurrent caller blocks in 100 ms sleep loop until a connection slot frees.

7. Known quirks and caveats

1. **Dual `sendData` paths** — pooling skips `AbstractTransport` wrapper; both set complete state manually in transport class.
 2. **Shared execution object** — lazy `DatabaseSqlExecution` singleton per transport instance.
 3. **Pooled wait loop** — indefinite spin until capacity; no explicit queue timeout.
 4. **Write Query replaces body** — prior stream discarded before SQLXML execute.
 5. **Callback listener rename** — only `CallbackListenerName` tracked (not a second service name).
 6. **Metadata restrictions** — default table pattern `%` still loads broad metadata unless tightened.
-

8. Operational recommendations

Goal	Guidance
Throughput	Enable pooling with connections near the DB pool limit
Consistency	Disable autocommit when SQLXML doc must be atomic
Large results	Use CallBack listener to fan out result streams

Goal	Guidance
Debugging	Enable SQLXML Logging temporarily; restrict metadata in prod
Simple statements	Write Query for ad hoc transports without SQLXML files

9. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
Database (SQL) transport	com/pilotfish/eip/modules/db/DatabaseSqlTransport.java
SQL execution / config	com/pilotfish/eip/modules/db/DatabaseSqlExecution.java
SQLXML handler	com/pilotfish/eip/util/SQLXMLHandler.java
Module JAR	war-pipeline/jars/eip-26R1.11/xcs-modules-db-1.0.1.jar

10. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> xcs-modules-db-1.0.1.jar

-> CFR decompile

-> Documents/Transports/26R1.11/PilotFish-Database-SQL-Transport-Reference-26R1.11.(md|pdf)

Deep-dive documentation from WAR decompile – Database (SQL) Transport – 26R1.11 – August 2026.